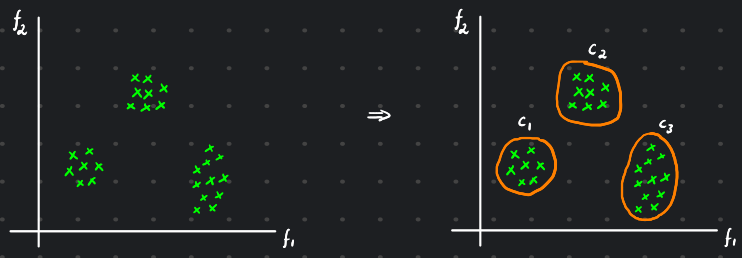
॥ श्रीगणेशाय नमः ॥

Clustering / k-means

Clustering

In clustering, we don't have the class labels for our data. The task is to group/cluster "similar" data points.



Similarity could mean that the points in a cluster are closer to each other than those in a different cluster.

Similarity is problem specific.

$D = \{x_i\}$

Applications of clustering

- E-commerce
  E-commerce companies use clustering to group similar customers based on their purchasing behavior (based on say how much they spend, how often they shop, where are they from, etc.). It could be that they offer specific deals to specific group of customers, or display targeted ads to groups who might have higher willingness to make use of those deals/ads.

- Image segmentation
  It refers to clustering similar pixels to separate an image into parts (say separating a sheep from the grass in the background). A classification algo could be used after segmentation to perform object detection.

- Consider the Amazon fine foods reviews dataset. There's the polarity of the labels: positive or negative. To label the set, manual labelling is really time consuming. In tasks like these, when manual labelling is time consuming or expensive, we could apply clustering first (based on say word similarity). We would then have to label the clusters, not the individual points.

- Medical imaging
  Cluster analysis can be used on PET scans to differentiate types of tissues in a 3-d image.

Metrics for clustering

We don't have the yi's to compare our predictions to.
How do we understand clustering results?



Intra-cluster: within a cluster
Inter-cluster: across/between clusters

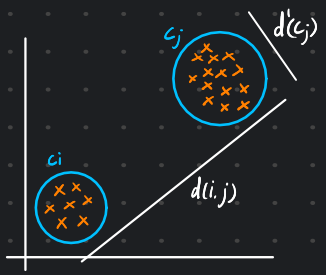
Intra-cluster distance is kept small, inter-cluster distance is kept large.

Dunn Index

$$D = \frac{\min_{i,j} d(i,j)}{\max_{k} d'(k)}$$

$\min_{i,j} d(i,j)$ } minimal inter cluster dist.
$\max_{k} d'(k)$ } maximal intra-cluster distance

d(i, j): distance between cluster i and cluster j
d'(k): intra-cluster distance of cluster k



d(i, j) = distance betn farthest points of Ci and Cj
d'(k) = distance betn farthest points within a cluster

k-means clustering

k: hyperparameter, # of clusters

Basic idea:
For a value of k, it groups the points into k clusters. For each cluster, there's a centroid. Every point belongs to exactly 1 cluster.
The centroid is the mean point to a cluster/set of points Si.

$$c_i = \frac{1}{n} \sum_{x_j \in S_i} x_j$$

Every point is assigned to the cluster with the nearest centroid.

Mathematical formulation

$D = \{x_1, x_2, \ldots, x_n\}$ Task:

k-centroids: $c_1, c_2, \ldots, c_k$ $\quad \forall\, x_i \in S_j$,
k-sets: $S_1, S_2, \ldots, S_k$ $\quad \forall\, i,j,\ S_i \cap S_j = \varphi$

i.e. to find

$$\underset{c_1, c_2, \ldots, c_k}{\operatorname{argmin}} \sum_{i=1}^{k} \sum_{x \in S_i} \|x - c_i\|^2 \quad \text{s.t.} \quad x \in S_i,\ S_i \cap S_j = \varphi$$

$\|x - c_i\|^2$ → sqd. distance of point $x \in S_i$ from centroid c_i

argmin → find centroids so as to minimize intracluster distance.

This is an NP-hard problem.

Lloyd's algorithm (naive k-means)

1. Initialization:
   Randomly pick k points from D as the centroids C1, C2, ..., Ck.

2. Assignment:
   For each point xi in D,
     select the nearest Cj
     add xi to set Sj

3. Recompute the centroids/ Update:
   Update the centroids:

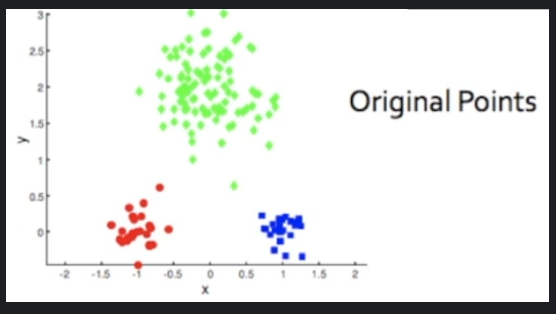
$$c_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

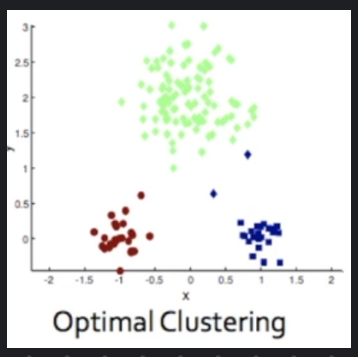
4. Repeat steps 2 and 3 until convergence, i.e. the centroids do not update anymore.

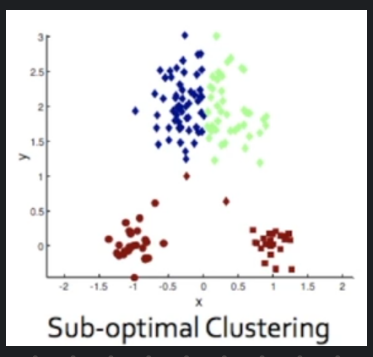
dist bet^n new and old centroids is very small.

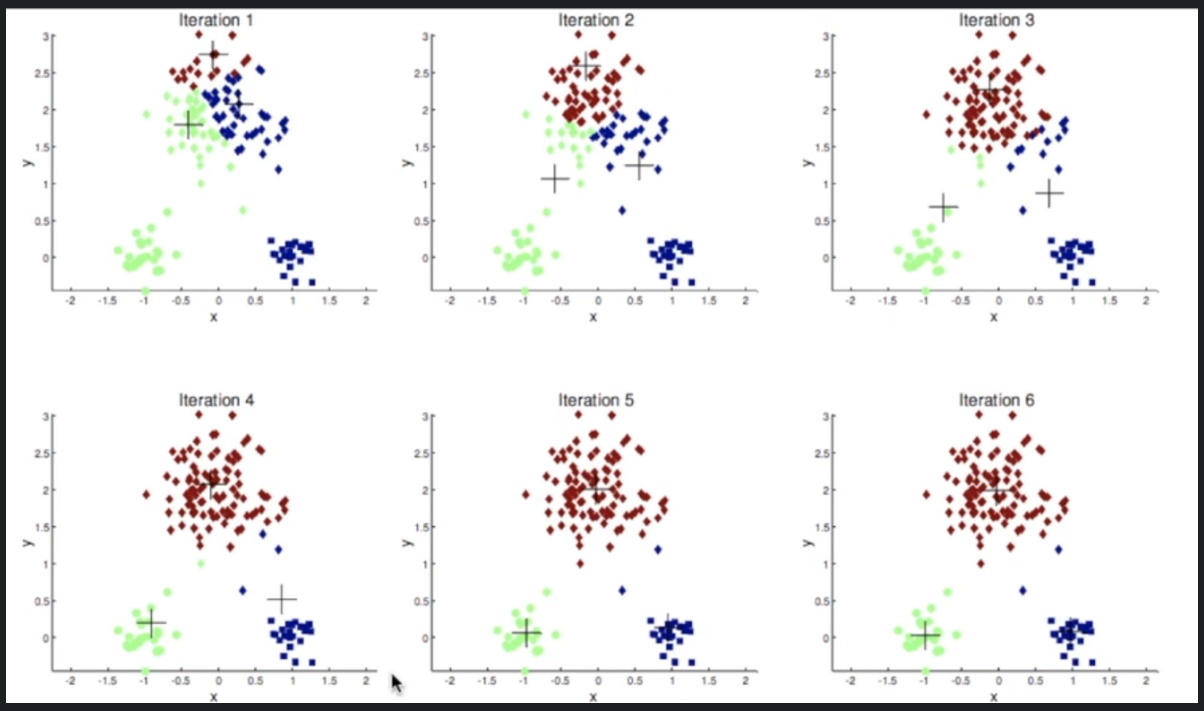
Initializing k-means

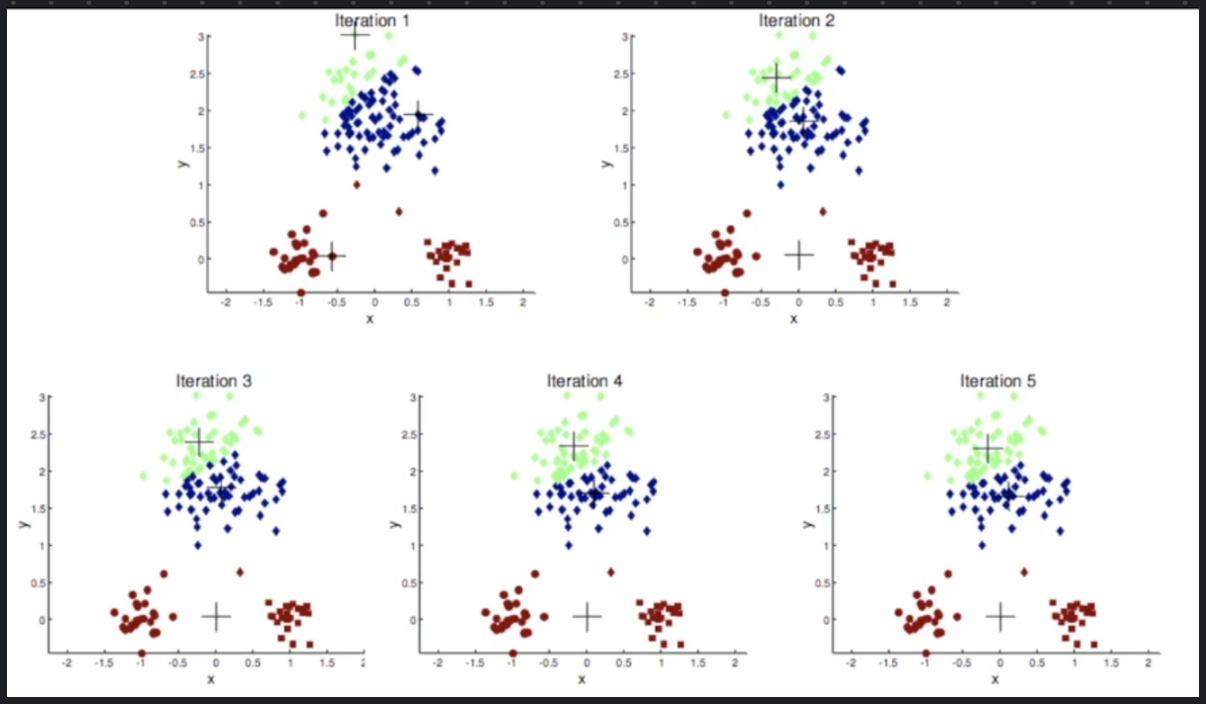
Initialization sensitivity:
The final clusters and centroids depend on the points picked initially.
So choosing points at random isn't necessarily a good idea.











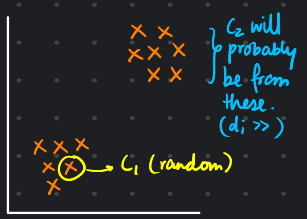
Dealing with initialization: 2 ways

1. Do multiple runs and select the clustering with the smallest error.

2. Select the original set of points by methods other than random. E.g. pick the points most distant from each other as cluster centers (k-means++).

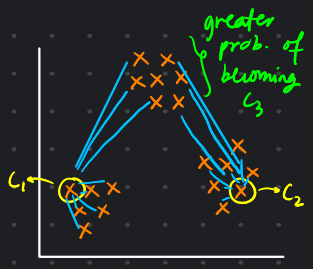
k-means ++

1. Pick the 1st centroid randomly from D.
2. $\forall x_i \in D$,
   $x_i \rightarrow dist^2(x_i, \text{nearest centroid})$ } A prob. distribu^n is created.



x_i	$d_i = \|x_i - c_1\|^2$
x_1	d_1
x_2	d_2
⋮	⋮
x_n	d_n

Pick a point from $D - \{C_1\}$ with probability proportional to d_i.
(more chances of picking points far from centroids)



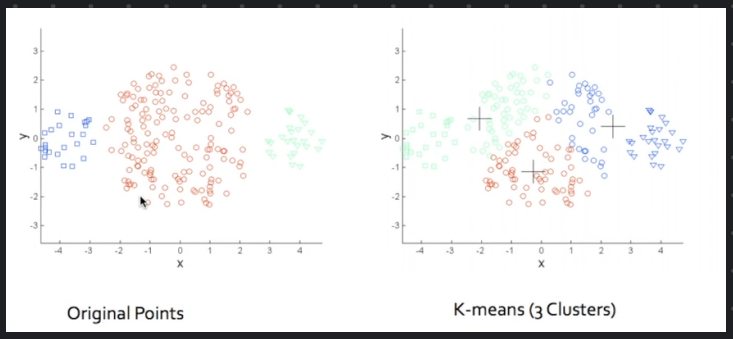
Say we want to pick 3 centroids (c_1, c_2, c_3).
and we choose c_1 and c_2.
For c_3,
we compute dist. of all points from the nearest centroid.
So we pick initial centroids far from each other.

Q: Why do this probabilistically? Why not just pick the farthest points?
→ outliers could be picked up as centroids.

K-means++ still gets affected by outliers, but this makes it a bit better.
So we just repeat this.

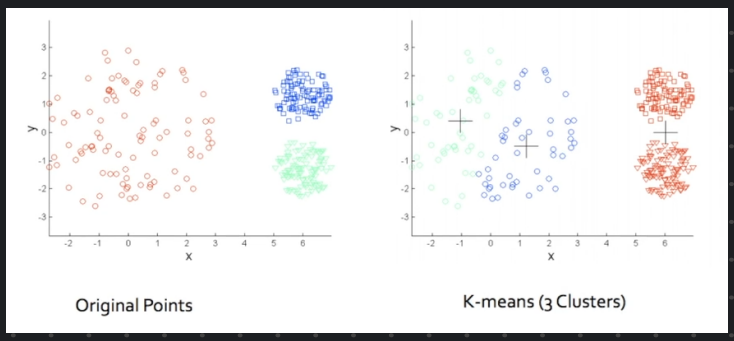
Limitations of k-means

1. Clusters of different sizes



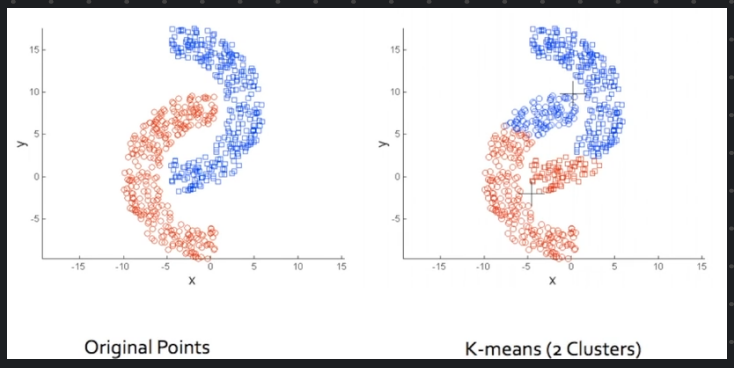
k-means tends to create clusters that are roughly of the same size.
It only checks the intra-cluster distances.

2. Clusters of different densities



k-means tries to stretch the clusters into those with similar densities.

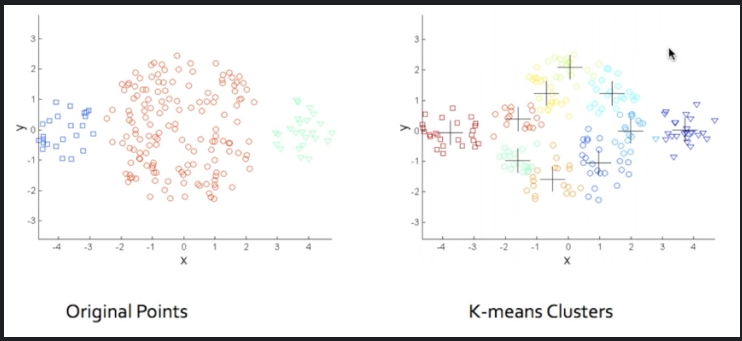
3. Non-globular cluster shapes (non-convex)

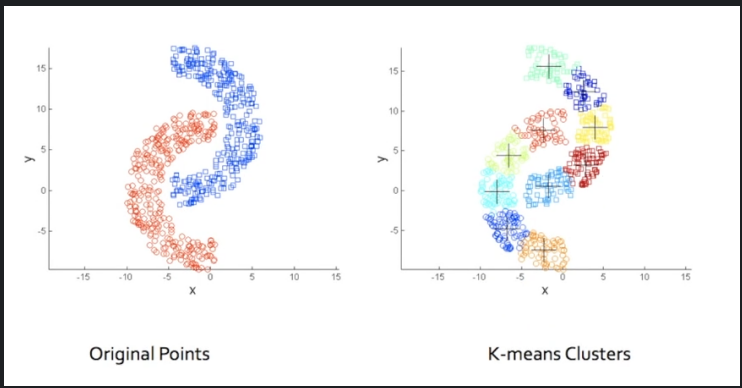


It tends to give concave sets.

Overcoming k-means limitations:

1. Use many clusters





Get more clusters than required and then put them together.
Putting those together is not too easy though.

k-medoids

Problem with k-means:
Centroids may not always be interpretable.
Say we have the Finefoods reviews data encoded using BoW. xi represents a review.
We compute the clusters C1, C2, ..., Ck.

A centroid would be a point 

→ can't figure it out.

Instead of giving Cj's computed using means, if we use some point that's a part of the dataset (say X_10):
In the example, we'll just get a particular review. We could determine if it is positive or negative.

So the solution is that each centroid is a datapoint. $C_i = x_j \in D$

Partitioning around medoids (PAM)

1. Initialization:
   Use ideas like k-means++.

2. Assignment:
   For each point xi in D,
   select the nearest medoid Cj
   add xi to set Sj

3. Recompute the medoids/ Update:
   Update the medoids:
   a. Swap each medoid with a non-medoid point.

Say k=2: 

b. If loss decreases, keep the swap. Else undo the swap.
↳ $\sum_{i=1}^{k} \sum_{x \in S_i} \|x - m_j\|^2$



Try all swaps and redo assignment.

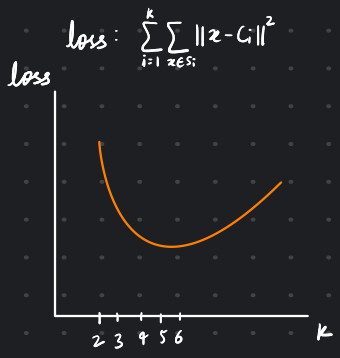
4. Repeat steps 2 and 3 until convergence, i.e. the medoids do not update anymore.

The only mathematical computation needed is to compute the distances between the points and the medoids.
So if we get pairwise distance or similarity values, we can use k-medoids.
k-medoids can be used with a similarity matrix.
So it allows kernelization trivially.

Determining the right k

- We could use domain knowledge.
- If we don't have any, we could use techniques like the elbow method.

Elbow method



plot the loss for various k values
and choose the one that minimizes it

In scikitlearn

sklearn.cluster.KMeans

class sklearn.cluster.**KMeans**(*n_clusters=8*, ***, *init='k-means++'*, *n_init='auto'*, *max_iter=300*, *tol=0.0001*, *verbose=0*, *random_state=None*, *copy_x=True*, *algorithm='lloyd'*) ¶ [source]